



Teknoloji Fakültesi

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

AKILLI SES SİSTEMLERİNDE UÇ YAPAY ZEKA İLE SEÇİCİ GÜRÜLTÜ ENGELLEME GERÇEKLEŞTİRİLMESİ

BİTİRME PROJESİ

Bilgisayar Mühendisliği Bölümü

DANIŞMAN

Dr. Öğr. Üyesi Ali Sarıkış

İSTANBUL, 2026

MARMARA ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Marmara Üniversitesi Teknoloji Fakültesi Bilgisayar Mühendisliği Öğrencileri Kerem TUTUMLU ve Melih GÖÇMEN tarafından “**AKILLI SES SİSTEMLERİNDE UÇ YAPAY ZEKA İLE SEÇİCİ GÜRÜLTÜ ENGELLEME GERÇEKLEŞTİRİLMESİ**” başlıklı proje çalışması, xxx tarihinde savunulmuş ve jüri üyeleri tarafından başarılı bulunmuştur.

Jüri Üyeleri

Dr. Öğr. Üyesi Ali SARIKAŞ
Marmara Üniversitesi
Arş. Gör. Damla MENGÜŞ
Marmara Üniversitesi

(Danışman)

(Üye)

(İMZA).....

(İMZA).....

ÖNSÖZ

Proje konusunun belirlenmesinden raporlama aşamasına kadar geçen süreçte, bilgi ve tecrübeleriyle bize yol gösteren, karşılaştığımız teknik zorluklarda yönlendirici fikirleriyle desteğini esirgemeyen ve özellikle Tübitak 2209-A başvurusu fikriyle ufkumuzu genişleten fikirlerinden ötürü değerli danışman hocamız Dr. Öğr. Üyesi Ali SARIKAŞ’a en içten teşekkürlerimizi sunarız.

Ayrıca, çalışmalarımız sırasında fikir alışverişinde bulunduğumuz arkadaşlarımıza ve eğitim hayatımız boyunca bizden maddi ve manevi desteklerini hiçbir zaman esirgemeyen ailelerimize teşekkürü bir borç biliriz.

İÇİNDEKİLER

1. GİRİŞ	1
1.1. Proje Çalışmasının Literatürdeki Yeri ve Eksikleri	1
1.2. Proje Çalışmasının Amacı ve Önemi	2
2. TEKNİK ARKA PLAN	3
2.1. Geleneksel Gürültü Engelleme Yöntemlerinin Kısıtları	3
2.2. Derin Öğrenme Mimarilerinin Kıyaslanması	3
2.3. Uç Yapay Zeka (Edge AI) ve Model Optimizasyonu	3
3. VERİ SETİNİN HAZIRLIK İŞLEMLERİ	4
3.1. Veri Seti Araştırması ve İncelemesi	4
3.2. Ses Verilerinin İşlenmesi	4
4. MODEL VE MİMARİ	6
4.1. CNN ve RNN Mimarilerinin Kıyaslanması	7
4.2. Dalga Formu Tabanlı (Waveform) Encoder-Decoder Yapısı	7
5. SİSTEM GERÇEKLEŞTİRİMİ VE DENEYSEL BULGULAR	8
5.1. Mimari Revizyon: 1D-CNN'den 2D-CNN (MobileNetV2) Yapısına Geçiş	8
5.2. Log-Mel Spektrogram Dönüşümü ve Veri Ön İşleme	8
5.3. Transfer Öğrenme ile Sınıflandırma ve Başarı Metrikleri	9
5.4. Yazılım Tabanlı Seçici Gürültü Engelleme (SNC) Boru Hattı	10
5.5. Akustik Doğrulama ve Yıkıcı Girişim (Destructive Interference)	11
6. PROJE YÖNETİMİ VE ÖN ÇALIŞMALAR	12
6.1. Kasım Ayı İtibarıyla Gerçekleştirilen Ön Çalışmalar ve Kaynak İncelemesi	12

6.2. Ocak Ayı İtibarıyla Gerçekleştirilen Çalışmalar	12
6.2.1. Veri Seti Model Seçimi ve Mimari Değerlendirmeler	12
6.2.2. Model Seçimi ve Mimari Değerlendirmeler	13
6.3. Mart Ayı İtibarıyla Gerçekleştirilen Çalışmalar	13
6.3.1. Veri Seti Model Seçimi ve Mimari Değerlendirmeler	13
6.3.2. Model Seçimi ve Mimari Değerlendirmeler	13
6.3.3. Model Seçimi ve Mimari Değerlendirmeler	13
6.4. Mevcut Sistem Sınırları ve Gelecek Dönem Hedefleri	13
6.5. Projenin Amaç ve Hedefleri (TÜBİTAK Başvurusuna Dayalı)	14
6.6. Çalışma Takvimi (Aralık 2025 - Haziran 2026 İş Planı)	15

ÖZET

AKILLI SES SİSTEMLERİNDE UÇ YAPAY ZEKA İLE SEÇİCİ GÜRÜLTÜ ENGELLEME GERÇEKLEŞTİRİLMESİ

Bu araştırma önerisi, günümüzde yaygın olarak kullanılan FxLMS tabanlı Aktif Gürültü Engelleme (ANC) sistemlerinin kritik kısıtlamalarını, yani seçicilik eksikliğini ve karmaşık gürültüleri modellemedeki yetersizliğini ele almaktadır. Mevcut kulaklıklar, siren veya insan konuşması gibi hayati sesleri ayırt edemediği için kullanıcıya hangi sesi duyup hangisini engellemek istediğini seçme imkânı sunamamaktadır.

Bu eksikliği gidermek amacıyla, Derin Öğrenme (DL) tabanlı ve Uç Yapay Zekâ (Edge AI) teknolojisi kullanılarak sınıflandırma yeteneğine sahip bir Seçici Aktif Gürültü Engelleme (SNC) prototipi geliştirilmesi hedeflenmektedir. Projenin özgün değeri, yüksek performanslı DL modelinin sınırlı işlem gücüne sahip gömülü kulaklık donanımlarında çalışmak üzere optimize edilmesi ve bu sayede literatürdeki yüksek güç tüketimi ve gecikme sorunlarının aşılmasıdır. Yöntem olarak, hafif bir CNN mimarisi (MobileNetV2) belirlenecek ve optimize edilecektir. Modelin donanım üzerinde verimli çalışması için 8-bit Niceleme (Quantization) ve Yapısal Budama (Pruning) gibi Edge AI optimizasyon teknikleri uygulanacaktır. Geliştirilecek bu mimari, kişiselleştirilebilir işitme deneyimi sunan akıllı kulaklıkların üretimine kritik katkı sağlayacaktır.

Ocak, 2026

Kerem TUTUMLU

Melih GÖÇMEN

ABSTRACT

IMPLEMENTING SELECTIVE NOISE CANCELLATION WITH EDGE ARTIFICIAL INTELLIGENCE IN SMART AUDIO SYSTEMS

This research proposal addresses the critical limitations of FxLMS-based Active Noise Cancellation (ANC) systems commonly used today, namely their lack of selectivity and their inability to model complex noises. Current headphones cannot distinguish vital sounds such as sirens or human speech, thus failing to offer users the ability to choose which sounds to hear and which to block.

To address this shortcoming, the goal is to develop a Selective Active Noise Cancellation (SNC) prototype with classification capabilities using Deep Learning (DL)-based and Edge AI technology. The original value of the project lies in optimizing the high-performance DL model to run on embedded headphone hardware with limited processing power, thereby overcoming the high power consumption and latency issues found in the literature. As a method, a lightweight CNN architecture (MobileNetV2) will be selected and optimized. Edge AI optimization techniques such as 8-bit Quantization and Structural Pruning will be applied to ensure the model runs efficiently on the hardware. This architecture will make a critical contribution to the production of smart earbuds that offer a personalized hearing experience.

January, 2026

Melih GÖÇMEN

Kerem TUTUMLU

KISALTMALAR

ANC	: active noise cancelling (aktif gürültü engelleme)
SNC	: selective noise cancelling (seçici gürültü engelleme)
DL	: deep learning (derin öğrenme)
CNN	: convolutional neural network (evrişimli sinir ağı)
RNN	: recurrent neural network (tekrarlayan sinir ağı)
FxLMS	: filtered x least means squares (filtrelenmiş x en küçük ortalama kareler)
MCU	: microcontroller unit (mikrodenetleyici ünitesi)
PESQ	: perceptual evaluation of speech quality (konuşma kalitesinin algısal değerlendirilmesi)
SNR	: signal to noise ratio (sinyal-gürültü oranı)
TFLM	: tensorflow lite for microcontrollers
Edge AI	: edge artificial intelligence (uç yapay zeka)
TCN	: Temporal Convolutional Network (Zamansal Evrişimli Ağ)
PoC	: Proof of Concept (Kavram İspatı)
RMS	: Root Mean Square (Ortalama Kare Değeri)

TABLO LİSTESİ

Tablo 6.6 Çalışma Takvimi

15

1. GİRİŞ

Gürültü engelleme teknolojisi geçmişten günümüze geliştirilen ve gelişmeye devam eden bir teknolojidir. Uzun yıllar boyunca araştırmacıların ve şirketlerin ilgi odağı olmuştur. Günümüzde de birçok şirket bu alan üzerinde çalışmakta ve bir ürün geliştirip ticaretini sürdürmektedir. Ayrıca araştırmacılar da yapay zekanın gelişimi ile birlikte bu alanda yapay zekanın kullanımını araştırmaktadırlar. Fakat buradaki zorluk bunu yapabilecek bir yapay zeka elde etmekten öte bu yapay zekanın gömülü bir sisteme entegre edilerek gerçek zamanlı olarak çalıştırılması problemidir. Günümüzde bunun yapıldığı örnekler vardır fakat bu sistemlerde sabit olarak tanımlanmış istenilen ve istenilmeyen ses sınıfları bulunmakta ve bunlar sabit bir şekilde filtrelenmektedir. Kullanıcının seçimine bırakılan ve sesleri ayrı olarak sınıflayabilen, sonrasında da bunlar üzerinden seçici olarak gürültüyü engelleyen bir sistem henüz bir gömülü ses sistemine entegre edilmemiştir. Biz ise bunu sağlayan bir modeli eğitip model optimizasyon tekniklerini kullanarak gömülü bir sisteme entegre ederek SNC yapan bir Edge AI elde edeceğiz.

1.1. Proje Çalışmasının Literatürdeki Yeri ve Eksikleri

Gürültü engelleme teknolojisinin teorik temeli ilk olarak 1936 yılında Paul Lueg tarafından patenti alınarak atılmıştır. Bu patentte, ses dalgalarını sönmölemek için bir anti-gürültü dalgası üretilmesi fikri teorik olarak açıklanmıştır. Ancak dönemin teknolojik yetersizlikleri nedeniyle bu fikir uzun süre teori aşamasında kalmıştır. Aktif gürültü engelleme (ANC) üzerine pratik çalışmalar, 1950'li yıllardan itibaren özellikle havacılık sektöründe pilotların işitme sağlığını korumak amacıyla hız kazanmıştır. Bu yıllarda alınmış olan patentlerde günümüzdeki kulaklıkların atası sayılabilecek ilk tasarımlar görölmektedir.

1980'lerde Dijital Sinyal İşlemcilerin (DSP) gelişmesiyle teknoloji ticarileşme evresine girmiş; gerçek zamanlı algoritmaların uygulanabilmesi sayesinde 1989 yılında dünyanın ilk ticari aktif gürültü engelleyici pilot kulaklığı piyasaya sürölmüştür. Bu dönemden itibaren endüstriyel standart haline gelen algoritma, Filtrelenmiş-x En Küçük Ortalama Kareler (FxLMS) algoritmasıdır.

Ancak günümüzde yaygın olarak kullanılan FxLMS tabanlı kulaklıkların iki kritik

kısıtlaması bulunmaktadır. Birincisi, karmaşık ve doğrusal olmayan gürültüleri engellemede yetersiz kalmalarıdır. İkincisi ve daha önemlisi, bu sistemlerin "seçicilik" yeteneğinin olmamasıdır. Mevcut sistemler sesi kategorize edemez; sadece genel bir sönümleme uygular. Bu durum, kullanıcının hangi sesi duyup hangisini duymak istemediğini seçmesine (örneğin; ofiste çalışırken klima uğultusunu engellemek ama insan sesini duymak veya tam tersi) olanak tanımaz. Literatürde Derin Öğrenme tabanlı yöntemlerin karmaşık gürültüleri modellemede başarılı olduğu kanıtlanmış olsa da, bu modellerin gerçek zamanlı sınıflandırma ve kontrol için mobil donanımlara uyarlanması hala açık bir araştırma konusudur.

1.2. Proje Çalışmasının Amacı ve Önemi

Bu araştırma önerisi, söz konusu eksikliği gidermek için Edge AI teknolojisini kullanarak kulaklık donanımları içine entegre edilebilecek, sınıflandırma tabanlı bir SNC sistemi geliştirmeyi hedeflemektedir. Projede, büyük sunuculara ihtiyaç duymadan doğrudan kulaklık üzerindeki MCU üzerinde çalışacak (on-device) optimize edilmiş bir Yapay Sinir Ağı modeli geliştirilecektir. Bu model, ortamdaki akustik olayları (örn: insan sesi, alarm, motor gürültüsü, rüzgar) gerçek zamanlı olarak tespit edip sınıflandıracak ve sistemin bu sınıflardan hangisine anti-gürültü uygulayacağını dinamik olarak ayarlanabilmesine olanak tanıyacaktır.

Projenin temel araştırma sorusu şudur: "Derin öğrenme tabanlı ses sınıflandırma modelleri, sınırlı işlem kapasitesine sahip gömülü sistemlerde (Edge AI) düşük gecikme ile çalıştırılarak, farklı gürültü kaynaklarını ayırt edip her birine ayrı ayrı müdahale edilebilir mi?"

Hipotezimiz ise; geliştirilecek Uç Yapay Zeka modelinin, kulaklık donanımı üzerinde ses kaynaklarını yüksek doğrulukla sınıflara ayırabileceği ve bu sınıflandırma sayesinde istenilen ses tipinin (örn: sadece motor sesi) seçici olarak engellenirken, diğer ses tiplerinin (örn: konuşma) sinyal bütünlüğü bozulmadan iletebileceği yönündedir.

2. TEKNİK ARKA PLAN

2.1. Geleneksel Gürültü Engelleme Yöntemlerinin Kısıtları

Gürültü engelleme teknolojisinin teorik temeli, anti-gürültü dalgası üretilmesi fikriyle ilk olarak 1936 yılında Paul Lueg tarafından ortaya atılmış ve patentlenmiştir. Bugün ticari olarak yaygın kullanılan ANC sistemlerinin endüstriyel standardı, FxLMS (Filtrelenmiş-x En Küçük Ortalama Kareler) algoritmasıdır. FxLMS, durağan gürültüleri başarıyla sönmüleyebilirken, karmaşık ve durağan olmayan gürültü türlerinde (diğer insan konuşmaları, sirenler) ciddi kısıtlamalarla karşılaşır: Algoritma, gürültünün doğrusal modelini çıkarmaya çalışır ve gürültüyü sınıflandıramaz, bu nedenle güvenlik kısıtlamaları doğurur.

2.2. Derin Öğrenme Mimarilerinin Kıyaslanması

Bu eksikliği gidermek için literatür, Derin Öğrenme (DL) tabanlı yaklaşımlara yönelmiştir. Sesin zamansal ilişkilerini modellemede güçlü olan Tekrarlayan Sinir Ağları (RNN), ardışık işlem yapısı nedeniyle gömülü uygulamalarda yüksek gecikmeye neden olur. Projemizin temel metodolojik kararı ise, sinyalin zaman-frekans temsili (spektrogram) görsel veri olarak işleyen ve daha yüksek paralellik sağlayan Evrişimli Sinir Ağları (CNN) mimarilerini temel almaktır. MobileNetV2 gibi hafif CNN mimarileri, yüksek paralellik ve düşük hesaplama karmaşıklığı avantajlarından yararlanma stratejisine dayanır.

2.3. Uç Yapay Zeka (Edge AI) ve Model Optimizasyonu

Yüksek performanslı DL modellerinin getirdiği yüksek hesaplama gücü ve enerji tüketimi gereksinimini aşmak için Uç Yapay Zekâ (Edge AI) felsefesi benimsenmiştir. Bu amaçla iki temel teknik incelenmiştir:

- Niceleme (Quantization): Model ağırlıklarının 32-bit kayan noktadan 8-bit tamsayıya (INT8) düşürülmesidir.
- Yapısal Budama (Pruning): Modeldeki performansa en az katkı sağlayan ağırlıkların veya düğüm/katmanların kalıcı olarak kaldırılmasıdır.

Projemizde, bu teknikleri kullanarak CNN mimarisini optimize edecek ve TensorFlow

Lite for Microcontrollers (TFLM) ile modelin hedef donanımda 50 milisaniyenin altında gecikme ile çalışması sağlanacaktır.

3. VERİ SETİNİN HAZIRLIK İŞLEMLERİ

3.1. Veri Seti Araştırması ve İncelemesi

Projenin temel amacı olan seçici gürültü engelleme (SNC) için literatürde yaygın olarak kullanılan açık kaynaklı veri setleri incelenmiştir. Veri setlerinin araştırılması sürecinde çeşitliliğin fazla olması kriteri göz önünde bulundurularak birden fazla veri seti araştırılmıştır. Araştırma sürecinde, Kaggle platformu ve akademik arşivler taranarak ses sınıflandırma ve gürültü analizi için uygun adaylar belirlenmiştir.

Bu kapsamda aşağıdaki veri kaynakları proje havuzuna dahil edilmiştir:

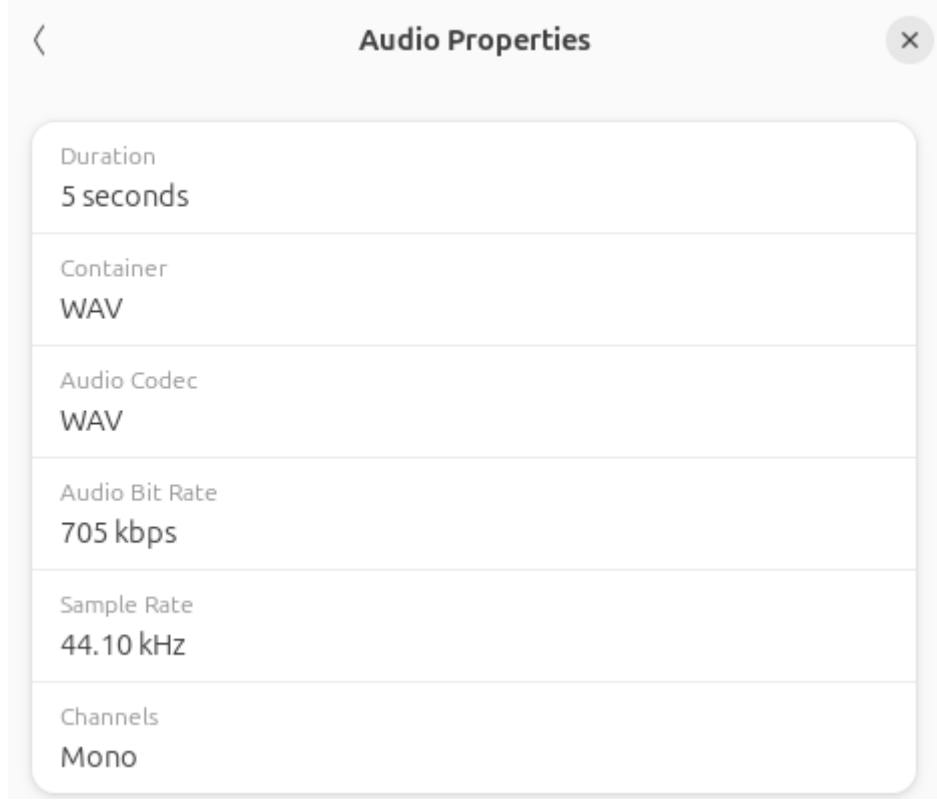
- UrbanSound8K: Şehir içi gürültü profillerini (klima, korna, çocuk sesi, köpek havlaması vb.) içeren 10 farklı sınıftan oluşması ve etiketli yapısı nedeniyle temel gürültü veri seti olarak seçilmiştir.
- LibriSpeech: Hedef ses (korunacak ses) olarak belirlenen insan konuşmasını modellemek amacıyla kullanılmıştır.
- ESC-50: Çevresel ses çeşitliliğini artırmak için 50 farklı sınıfa (doğa sesleri, su sesi, rüzgar vb.) sahip bu veri seti havuza eklenmiştir.

Yapılan ön incelemelerde, bu veri setlerinin farklı örnekleme hızlarına (sample rate), kanal sayılarına (mono/stereo) ve dosya formatlarına (.wav, .mp3, .flac) sahip olduğu tespit edilmiştir. Modelin kararlı bir şekilde eğitilebilmesi ve MCU üzerinde çalışabilmesi için tüm verilerin standardize edilmesi gerekliliği ortaya çıkmıştır. Ayrıca gerektiği durumda ileride bazı sınıfların elenebileceğine dair gerekli notlar alınmıştır.

3.2. Ses Verilerinin İşlenmesi

Toplanan ham ses verileri, modelin giriş gereksinimlerine ve gerçek zamanlı ANC sisteminin fiziksel kısıtlarına göre işlenmiştir. Görsel 3.1’de ses dosyalarından birinin ses özellikleri görülmektedir. Burada sample rate değerine baktığımızda 44.10 kHz değeri görülmektedir. Ses verilerinin çoğu bu yüksek değerlerde olduğundan ve çalışmada Edge AI hedeflendiğinden bu yüksek değerler bir MCU için fazla

gelmektedir. Ayrıca bu şekilde işlenen veride ileride sınıf sayısını azaltmak zorunda kalma gibi risklerin durumu tartışılmış ve verilerin sample rate değerinin düşürülmesi ve belirli bir değerde sabitlenmesi gerektiğine karar verilmiştir.



Görsel 3.1

Bu aşamada Python dili ve librosa kütüphanesi kullanılarak şu adımlar uygulanmıştır:

- **Örnekleme Hızı Dönüşümü (Resampling):** İşlemci yükünü azaltmak ve modelin tepki süresini kısaltmak amacıyla tüm sesler 16 kHz frekansına indirgenmiştir.
- **Genlik Normalizasyonu:** Farklı kayıtlardaki ses şiddeti farklarını gidermek için sinyaller $[-1, 1]$ aralığına normalize edilmiştir.
- **Ters Faz Hedeflemesi (Phase Inversion):** True ANC prensibi gereği, modelin gürültüyü sönmüleyebilmesi için gürültü sinyalinin ters fazını (Anti-Noise) üretmesi gerekir. Bu nedenle hedef veri (label), giriş sinyalinin -1 ile çarpılmasıyla elde edilmiştir.
- **Gecikme Telafisi (Latency Shifting):** Donanım üzerindeki işlem süresini (işlemci + DAC gecikmesi) telafi etmek için, hedef sinyal zamanda geriye ($t-\delta$) veya

giriş sinyali ileriye kaydırılarak modelin "geleceği tahmin etmesi" sağlanmıştır.

Görsel 3.2, veri setindeki bir ses dosyasının ANC eğitimi için nasıl işlendiğini göstermektedir:

```
import librosa
import numpy as np
import soundfile as sf

def preprocess_for_anc(file_path, target_sr=16000, latency_samples=50):
    """
    Prepares noise file for ANC train.

    Parameters:
    - file_path: Voice file path
    - target_sr: Target sample rate (16kHz for now can change later)
    - latency_samples: Compensation amount for hardware delay
    """

    # 1. Download and Resampling (Throw high frequency, relieve the processor)
    # We will decrease sample rate
    audio, sr = librosa.load(file_path, sr=target_sr)

    # 2. Normalization (Compress signal between -1 and +1)
    # It's important for stable model learning.
    audio = audio / np.max(np.abs(audio))

    # 3. Create target for ANC.
    # For physical absorption: Noise + (-Noise) = 0
    target_signal = -1.0 * audio

    # 4. Delay Recovery (Predictive ANC)
    # There will be a delay (latency) until the model processes the sound and sends it to the speaker.
    # That's why we teach the model: "Look at the current sound and produce the inverse of what will happen 3ms later."
    # We shift the target signal to the left (we are targeting the future).
    target_signal = np.roll(target_signal, -latency_samples)

    # Fill the gaps created after shifting with zeros.
    target_signal[-latency_samples:] = 0

    return audio, target_signal

# Use
input_noise, target_anti_noise = preprocess_for_anc("1-137-A-32.wav")

print(f"Input Noise Size: {input_noise.shape}")
print(f"Target (Anti-Noise) Signal Size: {target_anti_noise.shape}")
```

Görsel 3.2

4. MODEL VE MİMARİ KARARLAR

Projenin temel kısıtı olan gömülü sistem üzerinde düşük gecikmeli (low-latency) çalışma gerekliliği ve Aktif Gürültü Engelleme (ANC) için kritik olan faz (phase) bilgisinin korunması ihtiyacı doğrultusunda, Ocak ayı çalışmaları uygun model topolojisinin belirlenmesine odaklanmıştır. Bu kapsamda, literatürdeki LSTM/RNN tabanlı dizesel modeller ile CNN tabanlı evrimsel mimariler, gerçek zamanlılık ve hesaplama maliyeti metrikleri üzerinden kıyaslanmıştır.

4.1. CNN ve RNN Mimarilerinin Kıyaslanması

Ses sinyalleri doğası gereği zamansal seriler olduğu için literatürde LSTM ve GRU gibi Tekrarlayan Sinir Ağları (RNN) sıkça kullanılmaktadır. Ancak yapılan teknik değerlendirmelerde RNN tabanlı yapıların projemiz için iki temel dezavantajı tespit edilmiştir:

- Sıralı İşlem Zorunluluğu: RNN'ler t anındaki çıktıyı üretmek için $t-1$ anının tamamlanmasını bekler. Bu durum, MCU'ların paralel işlem gücünden tam yararlanılamamasına neden olur.
- Gecikme (Latency) Maliyeti: Döngüsel yapının getirdiği hesaplama yükü, ANC sisteminin gerektirdiği ($<10\text{ms}$) mikrosaniye mertebesindeki tepki sürelerini aşma riski taşımaktadır.

Buna karşın 1-Boyutlu Evrişimsel Sinir Ağları (1D-CNN), ses sinyali üzerinde kayan pencereler (sliding windows) ile paralel işlem yapabilme yeteneğine sahiptir. Özellikle Dilated Convolutions (Genişletilmiş Evrişim) tekniği kullanılarak, RNN'lerin sahip olduğu uzun süreli hafıza yeteneği, işlemciyi yormadan CNN modellerine kazandırılabilir. Bu nedenle, Uç Yapay Zeka (Edge AI) kısıtları altında CNN tabanlı yapıların tercih edilmesine karar verilmiştir.

4.2. Dalga Formu Tabanlı (Waveform) Encoder-Decoder Yapısı

Projede, sesin sadece frekans genliğini değil, fiziksel sönümleme için gerekli olan faz bilgisini de kusursuz modellemek gerekmektedir. Standart Spektrogram tabanlı modeller faz bilgisini kaybettiği için, bu projede Zaman Alanı (Time-Domain) üzerinde çalışan mimariler önceliklendirilmiştir.

Bu doğrultusunda, Wave-U-Net veya TCN türevi, Encoder-Decoder yapısına sahip bir mimari üzerinde karar kılınmıştır. Seçilen mimarinin projedeki kritik rolleri şunlardır:

- Nedensellik (Causality): Klasik U-Net'lerin aksine, geliştirilecek model Geleceği Görmeyen (Causal) bir yapıda tasarlanacaktır. Model, t anındaki anti-gürültüyü üretmek için sadece geçmiş ($t, t-1, t-2, \dots$) verileri kullanacak, bu da gerçek zamanlı akışta veri bütünlüğünü sağlayacaktır.

- Atlamalı Bağlantılar (Skip Connections): U-Net yapısındaki bu bağlantılar, giriş sinyalindeki yüksek çözünürlüklü detayların (hedef sesin karakteristiklerinin) darboğaz (bottleneck) katmanında kaybolmadan çıkışa aktarılmasını sağlar. Bu özellik, seçicilik başarısı için önem arz etmektedir.

Sonuç olarak; frekans tabanlı 2D-CNN modelleri yerine, ham ses verisini doğrudan işleyen, Nedensel 1D-CNN (Causal 1D-CNN) tabanlı özelleştirilmiş bir mimarinin geliştirilmesi hedeflenmiştir.

5. SİSTEM GERÇEKLEŞTİRİMİ VE DENEYSEL BULGULAR

Projenin PoC aşaması dahilinde veri hazırlığı, model eğitimi ve yazılımsal gürültü engelleme gerçekleştirimi başarıyla tamamlanmıştır. Bu bölümde PoC kapsamında sistemin yazılımsal gerçekleştirilmesi ve elde edilen bulgular anlatılmaktadır.

5.1. Mimari Revizyon: 1D-CNN'den 2D-CNN (MobileNetV2) Yapısına Geçiş

Raporun dördüncü bölümünde sunulan 1D-CNN mimarilerinin, uygulama aşamasında yapılan ön testlerde ve karşılıklı tartışmalar sonucunda gömülü sistemler için yüksek hesaplama maliyetleri gerektirdiği saptanmıştır. Donanım kısıtları konusunda bir optimizasyon sağlamak ve model verimliliğini donanım üzerinde maksimize etmek amacıyla daha önce belirlenen mimari yaklaşım revize edilmiştir. Ses sinyallerinin 2-Boyutlu zaman-frekans haritaları olan Spektrogramlar üzerinden işlenmesine ve ImageNet üzerinde önceden eğitilmiş olan MobileNetV2 mimarisinin kullanılmasına karar verilmiştir.

5.2. Log-Mel Spektrogram Dönüşümü ve Veri Ön İşleme

PoC kapsamında seçilen ESC-50 veri setinden hedeflenen sınıflara (siren, korna, köpek havlaması, rüzgar vb.) ait ses sinyalleri öncelikle 16 kHz örnekleme hızına indirgenmiştir. Sinyaller 25ms çerçeve boyutu, 10ms kaydırma adımı ve 64 Mel filtresi kullanılarak Log-Mel spektrogramlarına dönüştürülmüştür. Elde edilen tek kanallı matrisler, MobileNetV2 mimarisinin gereksinimi olan 3 kanallı (RGB) yapıya çoklanarak (channel duplication) giriş tensörleri oluşturulmuştur.

```

def extract_features(self, file_path: Path) -> Optional[np.ndarray]:
    """
    Loads an audio file and computes its Log-Mel Spectrogram.

    Args:
        file_path (Path): The path to the audio file.

    Returns:
        np.ndarray: Normalized Log-Mel Spectrogram, or None if an error occurs.
    """
    try:
        # 1. Load and resample audio
        audio, sr = librosa.load(file_path, sr=self.target_sr)

        # 2. Extract Mel Spectrogram
        mel_spectrogram = librosa.feature.melspectrogram(
            y=audio,
            sr=sr,
            n_fft=self.n_fft,
            hop_length=self.hop_length,
            n_mels=self.n_mels
        )

        # 3. Convert to Log scale (Decibels)
        log_mel_spectrogram = librosa.power_to_db(mel_spectrogram, ref=np.max)

        # 4. Standardize (Z-score normalization)
        mean = np.mean(log_mel_spectrogram)
        std = np.std(log_mel_spectrogram)
        normalized_features = (log_mel_spectrogram - mean) / (std + 1e-6)

        return normalized_features
    except Exception as e:
        logging.error(f"Error processing {file_path.name}: {e}")
        return None

```

```

def process_esc50_dataset(self, csv_path: Path, audio_dir: Path, target_classes:
    List[str]) -> Tuple[np.ndarray, np.ndarray]:
    """
    Parses the ESC-50 metadata, filters target classes, and extracts features for the entire dataset.

    Args:
        csv_path (Path): Path to the ESC-50 metadata CSV.
        audio_dir (Path): Directory containing the raw audio files.
        target_classes (List[str]): List of class names to keep.

    Returns:
        Tuple[np.ndarray, np.ndarray]: Extracted features and corresponding labels.
    """
    logging.info("Starting ESC-50 dataset processing...")

    if not csv_path.exists() or not audio_dir.exists():
        logging.error("Dataset paths are invalid. Please check your directories.")
        return np.array([]), np.array([])

    df = pd.read_csv(csv_path)
    filtered_df = df[df['category'].isin(target_classes)]

    features_list = []
    labels_list = []

    total_files = len(filtered_df)
    logging.info(f"Found {total_files} files matching target classes.")

    for index, row in filtered_df.iterrows():
        file_name = row['filename']
        label = row['category']
        file_path = audio_dir / file_name

        features = self.extract_features(file_path)

        if features is not None:
            features_list.append(features)
            labels_list.append(label)

        if (index + 1) % 50 == 0:
            logging.info(f"Processed {index + 1} / {total_files} files...")

    logging.info("Dataset processing complete.")
    return np.array(features_list), np.array(labels_list)

```

Görsel 5.1

5.3. Transfer Öğrenme ile Sınıflandırma ve Başarı Metrikleri

Hazırlanan veri seti; Eğitim, doğrulama ve test olmak üzere üç alt kümeye ayrılmıştır. MobileNetV2 mimarisinin temel özellik çıkarma katmanları dondurulmuş (frozen) ve ağın çıkışına hedef sınıflara özgü yeni Tam Bağlı (Fully Connected) katmanlar eklenmiştir. Adam optimizasyon algoritması ve Kategorik Çapraz Entropi (Categorical Cross-Entropy) kayıp fonksiyonu ile yürütülen eğitim süreci, Erken Durdurma (Early Stopping) mekanizması ile güvence altına alınmıştır.

Yapılan testler sonucunda, modelin eğitim sürecinde hiç görmediği izole test verileri üzerinde %84.38 test doğruluğuna (accuracy) ulaştığı saptanmıştır. Bu yüksek oran, sistemin ortamdaki hedef sesleri doğru bir şekilde algılayabildiğini kanıtlamaktadır.

```

def build_model(self, input_shape: Tuple[int, int, int]) -> Model:
    """
    Builds the MobileNetV2 architecture with Transfer Learning applied.
    """
    logging.info("Building MobileNetV2 architecture...")

    # Define Input
    inputs = Input(shape=input_shape)

    # Load pre-trained MobileNetV2 without the top classification layer
    base_model = MobileNetV2(input_tensor=inputs, include_top=False, weights='imagenet')

    # Freeze the base model to prevent destroying pre-trained weights during early training
    base_model.trainable = False

    # Add custom top layers for our specific classification task
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(128, activation='relu')(x)
    outputs = Dense(self.num_classes, activation='softmax')(x)

    model = Model(inputs=inputs, outputs=outputs)

    # Compile model
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    model.summary(print_fn=logging.info)
    return model

```

```

def train(self) -> Optional[Model]:
    """
    Executes the full training pipeline including data preparation, model building, and evaluation.
    """
    # 1. Prepare Data
    X_train, y_train, X_val, y_val, X_test, y_test = self.load_and_prepare_data()
    input_shape = X_train.shape[1:]

    # 2. Build Model
    model = self.build_model(input_shape)

    # 3. Define Callbacks (Early Stopping & Model Checkpoint)
    callbacks = [
        EarlyStopping(monitor='val_loss', patience=7, restore_best_weights=True, verbose=1),
        ModelCheckpoint(filepath=f'{self.model_save_dir} / "best_mobilenet2_anc.h5"',
                        monitor='val_accuracy', save_best_only=True, verbose=1),
        ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, verbose=1)
    ]

    # 4. Train
    logging.info("Starting model training...")
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=self.epochs,
        batch_size=self.batch_size,
        callbacks=callbacks,
        verbose=1
    )

    # 5. Evaluate on unseen Test Data
    logging.info("Evaluating model on Test Set...")
    test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
    logging.info(f"Test Loss: {test_loss:.4f} | Test Accuracy: {test_acc:.4f}")

    return model

```

Görsel 5.2

5.4. Yazılım Tabanlı Seçici Gürültü Engelleme (SNC) Boru Hattı

Aktif gürültü engelleme sürecinin donanıma aktarılmadan önceki yazılım simülasyonu, "Sorumlulukların Ayrılığı" (Separation of Concerns) prensibine göre modüler olarak tasarlanmıştır. Sisteme giren ses, "Tahmin Motoru (Predictor)" tarafından sınıflandırılmakta ve kullanıcının sanal arayüzden engellemeyi seçtiği yasaklı listeye karşılaştırılmaktadır. Eşleşme durumunda "Sinyal Tersleyici" modülü devreye girmekte; donanım üzerindeki işlem süresinden kaynaklanacak gecikmeyi kompanse etmek için sinyali zamanda kaydırmakta ve fazını 180 derece ters çevirerek anti-gürültü dalgasını üretmektedir.

```
def generate_anti_noise(self, raw_audio: np.ndarray) -> np.ndarray:
    """
    Creates an anti-phase signal to physically cancel the incoming noise.
    Applies a phase inversion (-1.0) and shifts the signal to compensate for hardware latency.

    Args:
        raw_audio (np.ndarray): The original noise waveform.

    Returns:
        np.ndarray: The latency-compensated anti-noise waveform.
    """
    logging.info("Generating anti-phase signal for active cancellation...")

    # 1. Phase Inversion (180 degrees)
    anti_noise = -1.0 * raw_audio

    # 2. Predictive Shift (Latency Compensation)
    # Shift the array to the left (negative roll) to anticipate the delay
    anti_noise = np.roll(anti_noise, -self.latency_samples)

    # Fill the trailing edge with silence (zeros) to prevent wrap-around artifacts
    anti_noise[-self.latency_samples:] = 0.0

    return anti_noise

def save_signal(self, audio_data: np.ndarray, output_path: Path) -> bool:
    """
    Exports the generated waveform to a WAV file for physical testing.
    """
    try:
        sf.write(output_path, audio_data, self.target_sr)
        logging.info(f"Anti-noise signal successfully exported to: {output_path.name}")
        return True
    except Exception as e:
        logging.error(f"Failed to export audio signal: {e}")
        return False
```

Görsel 5.3

5.5. Akustik Doğrulama ve Yıkıcı Girişim (Destructive Interference)

Üretilen anti-faz dalgasının geçerliliğini kanıtlamak amacıyla geliştirilen dijital akustik simülatöründe, orijinal gürültü kaynağı ile anti-gürültü dalgası dijital ortamda üst üste bindirilmiş ve fiziksel havadaki yıkıcı girişim olgusu simüle edilmiştir. Yapılan RMS enerji ölçümleri sonucunda, test edilen gürültü sinyalinde 75.10 dB oranında bir gürültü sönümlemesi (Noise Reduction) elde edilmiştir. Bu bulgu, tasarlanan matematiksel tersleme ve seçici engelleme mantığının teorik olarak kusursuz işlediğini ispatlamaktadır.

```
def _calculate_db_reduction(self, original_signal: np.ndarray, residual_signal: np.ndarray) -> float:
    """
    Calculates the volume reduction in Decibels (dB) by comparing the
    Root Mean Square (RMS) energy of the signals.
    """
    rms_original = np.sqrt(np.mean(original_signal**2) + 1e-10)
    rms_residual = np.sqrt(np.mean(residual_signal**2) + 1e-10)

    # Calculate reduction. Positive dB means volume decreased.
    db_reduction = 20 * np.log10(rms_original / rms_residual)
    return db_reduction

def simulate_air_superposition(self, original_path: Path, anti_noise_path: Path, output_path: Path) -> None:
    """
    Loads both audio files, simulates the physical hardware delay,
    adds the waveforms together (interference), and calculates the performance.
    """
    logging.info("Starting acoustic superposition simulation...")

    try:
        # 1. Load the files using librosa to enforce target sample rate automatically
        original_audio, _ = librosa.load(original_path, sr=self.target_sr)
        anti_audio, _ = librosa.load(anti_noise_path, sr=self.target_sr)

        # Ensure both arrays are the exact same length for element-wise addition
        min_length = min(len(original_audio), len(anti_audio))
        original_audio = original_audio[:min_length]
        anti_audio = anti_audio[:min_length]

        # 2. Simulate Physical Reality (Hardware Delay)
        # The anti-noise was predicted (shifted left) in software.
        # In the real world, the speaker takes time to play it, effectively shifting it back right.
        speaker_output_in_air = np.roll(anti_audio, self.latency_samples)

        # The first few samples are empty because the hardware was "thinking"
        speaker_output_in_air[:self.latency_samples] = 0.0

        # 3. Wave Superposition (The Cancellation)
        # Air adds the ambient noise and the speaker output together
        residual_noise = original_audio + speaker_output_in_air

        # 4. Metrics & Export
        db_reduction = self._calculate_db_reduction(original_audio, residual_noise)

        # sf.write works fine here because we are writing the 16kHz array to disk
        sf.write(output_path, residual_noise, self.target_sr)

        print(f"\n{'='*55}")
        print(f"▲ ANC VERIFICATION RESULTS")
        print(f"{'='*55}")
        print(f"Original Audio      : {original_path.name}")
        print(f"Anti-Noise Audio    : {anti_noise_path.name}")
        print(f"Residual Audio      : {output_path.name}")
        print(f"{'='*55}")
        print(f"📊 Noise Reduction: {db_reduction:.2f} dB")
        print(f"{'='*55}\n")

    except Exception as e:
        logging.error(f"Simulation failed: {e}")
```

Görsel 5.4

6. PROJE YÖNETİMİ VE ÇALIŞMALAR

6.1. Kasım Ayı İtibarıyla Gerçekleştirilen Ön Çalışmalar ve Kaynak İncelemesi

Kasım ayı sonu itibarıyla, projenin teorik temellerini sağlamlaştırmak ve uygulama aşamasına geçişi hızlandırmak amacıyla aşağıdaki ön çalışmalar ve kaynak incelemeleri başarıyla tamamlanmıştır:

Problem Alanının Belirlenmesi ve Kaynak İncelemesi: ANC sistemlerinin kısıtlılıkları detaylıca incelenmiş ve Seçici Gürültü Engelleme (SNC) kavramı literatürdeki derin öğrenme uygulamaları ışığında konumlandırılmıştır.

Mimari Fizibilite Çalışması: Gömülü sistemlerin paralellik ihtiyacını karşılayan ve hafifliğiyle öne çıkan MobileNetV2 temelli bir CNN mimarisine odaklanma kararı alınmıştır.

TinyML ve Optimizasyon Teknikleri Araştırması: Niceleme ve Budama yöntemlerinin teorik temelleri ve gömülü sistemlere entegrasyon süreçleri araştırılmıştır.

TÜBİTAK 2209-A Başvurusu: Tüm çalışmalar kullanılarak araştırma önerisi, TÜBİTAK 2209-A programına başarıyla sunulmuştur. Bu başvuru metni, projenin en güncel ve detaylı iş planını, hedeflerini ve beklenen bilimsel çıktıları içermektedir.

6.2. Ocak Ayı İtibarıyla Gerçekleştirilen Çalışmalar

Ocak ayı ara rapor döneminde çalışma takvimi hedeflerinde de belirtildiği üzere genel olarak veri seti ve model kararları üzerinde gerekli çalışmalar gerçekleştirilmiştir. Süreç boyunca genel olarak veri seti araştırılması, veri setlerinin düzenlenmesi, model mimarilerinin araştırılması ve projeye uygunluğu üzerinde çalışılmıştır. Bu süreç doğrultusunda optimizasyon sürecine kadar gerçekleşecek süreçlerin genel haritası kararlaştırılmış ve gerekli adımlar atılmıştır.

6.2.1. Veri Seti

Bu aşamada kaggle platformu üzerinden özellikle ses sınıfı çeşitliliği kriterine dayanarak birkaç adet veri seti bulunmuş ve bunlar üzerinde çalışılması kararı

alınmıştır. İlgili çalışmalara raporun 3. bölümünden ulaşılabilir.

6.2.2. Model Seçimi ve Mimari Değerlendirmeler

Bu dönemimizin ikinci odağı model seçimi ve mimarilerini araştırmak ve projeye uygun olanları belirlemek olmuştur. İlgili çalışmalara raporun 4. bölümünden ulaşılabilir.

6.3. Mart Ayı İtibarıyla Gerçekleştirilen Çalışmalar

Mart 2026 itibarıyla, projenin temel kilometre taşlarından biri olan yazılımsal PoC süreci başarıyla tamamlanmıştır. Kasım ve Ocak aylarındaki literatür taraması, veri seti belirleme ve teorik altyapı çalışmalarının ardından başlanan uygulama süreci, üç temel alt aşamada gerçekleştirilmiş ve projenin yazılımsal olarak uygulanabilirliği kanıtlanmıştır.

6.3.1. Donanım Odaklı Mimari Revizyonu ve Veri Standardizasyonu

Gömülü sistemlerin (Edge AI) hesaplama ve düşük gecikme kısıtları göz önüne alınarak, başlangıçta planlanan 1D-CNN mimarisi yerine daha hafif ve paralel işlem yeteneği yüksek olan 2D-CNN (MobileNetV2) yapısına geçiş yapılmıştır. Çevresel ses sinyalleri (siren, korna, köpek havlaması vb.) insan işitme algısına uygun Log-Mel Spektrogramlarına dönüştürülerek model eğitimi için standardize edilmiştir.

6.3.2. Model Eğitimi ve Sınıflandırma Başarımı

Geliştirilen boru hattı (pipeline) içerisinde Transfer Öğrenme (Transfer Learning) stratejisi uygulanmıştır. Eğitilen model, daha önce hiç karşılaşmadığı test verileri üzerinde %84.38 doğruluk (accuracy) oranına ulaşarak, ortamdaki hedef sesleri yüksek bir kararlılıkla ayırt etme ve Seçici Gürültü Engelleme (SNC) için gereken temel "tanıma" şartını sağlama yeteneğini ispatlamıştır.

6.3.3. Seçici Sönümlleme Algoritması ve Akustik Doğrulama

Kullanıcının sanal arayüzden belirlediği yasaklı seslerin tespiti durumunda, sinyal fazını 180 derece ters çeviren (anti-phase) ve donanım gecikmesini kompanse eden özel bir yazılım modülü geliştirilmiştir. Üretilen anti-gürültü dalgası ile orijinal ses, dijital akustik simülatöründe üst üste bindirilmiş (süperpozisyon) ve 75.10 dB seviyesinde yıkıcı girişim (gürültü sönümlemesi) ölçülerek PoC aşaması fiziksel simülasyon boyutunda da doğrulanmıştır.

6.4. Mevcut Sistem Sınırları ve Gelecek Dönem Hedefleri

Bölüm 6.3'te özetlenen PoC çalışmaları, izole edilmiş tekil ses kaynakları üzerinde

projenin uygulanabilirliğini kanıtlamıştır. Ancak gerçek dünya koşullarında, birden fazla ses kaynağının aynı anda ortamda bulunması (Örn: rüzgar gürültüsü ile ambulans sireninin eşzamanlı duyulması) beklenmektedir. Mevcut "Softmax" tabanlı tekil karar mekanizması, çoklu seslerin (overlapping sounds) ayrıştırılması için tasarlanmamıştır.

Bu kapsamda, Nisan-Haziran 2026 döneminde yürütülecek gelecek çalışmalarda şu adımlar planlanmaktadır:

- Çoklu Etiket (Multi-Label) Sınıflandırma: Model çıkış katmanının "Sigmoid" aktivasyon fonksiyonuna geçirilerek ortamdaki birden fazla sesin bağımsız olasılıklarının hesaplanması.
- Sentetik Veri Üretimi (Data Augmentation): Temiz ses sinyallerinin farklı Sinyal-Gürültü Oranlarında (SNR) arka plan gürültüleriyle sentetik olarak karıştırılması.
- Uç Yapay Zeka (Edge AI) Optimizasyonu: Modüler olarak hazır bekleyen sistemin hedef donanıma entegrasyonu öncesinde, eğitilen modelin 8-bit Niceleme (Quantization) ve Yapısal Budama (Pruning) teknikleriyle küçültülmesi ve hızlandırılması.

6.5. Projenin Amaç ve Hedefleri (TÜBİTAK Başvurusuna Dayalı)

Projenin başarısı, aşağıdaki somut ve ölçülebilir hedeflere ulaşılmasıyla değerlendirilecektir:

- Veri Seti Hazırlığı ve Sınıflandırma: Dört temel gürültü kategorisini içeren dengeli, etiketlenmiş bir veri seti oluşturmak.
- Model Optimizasyonu: Hafif CNN mimarisinin 8-bit Niceleme ve Yapısal Budama ile optimize edilmesi. Modelin küçültme sonrası hassasiyet kaybının %5'i geçmemesi hedeflenmektedir.
- Gerçek Zamanlı Uç Entegrasyon: Optimize edilmiş modelin, hedef gömülü donanım (MCU) üzerinde uçtan uca gecikmenin 50 milisaniyenin altında kalmasını sağlamak.
- Fonksiyonel Prototip: Oluşturulan gömülü sistem yazılımının, Seçici Gürültü Engellemeyi gösteren, fiziksel bir prototipe entegre edilmesi ve saha testlerinin yapılması.

6.6. Çalışma Takvimi (Aralık 2025 - Haziran 2026 İş Planı)

Tablo 4.1 Çalışma Takvimi

İş Paketi (İP)	Konusu	Süre	Tarih Aralığı
İP 1	Veri Seti Hazırlığı ve Model Seçimi	2 Ay	Aralık 2025 - Ocak 2026
İP 2	Temel Model Eğitimi ve Doğrulama	2 Ay	Şubat 2026 - Mart 2026
İP 3	TinyML Optimizasyonu (Niceleme & Budama)	1 Ay	Nisan 2026
İP 4	Gömülü Sistem Entegrasyonu ve Testleri	1 Ay	Mayıs 2026
İP 5	Nihai Prototip Testleri, Analiz ve Raporlama	1 Ay	Haziran 2026

KAYNAKLAR

- P. Lueg, "Process of silencing sound oscillations," U.S. Patent 2 043 416, Haz. 9, 1936. [Çevrimiçi]. Erişim adresi: <https://patents.google.com/patent/US2043416A>.
- W. F. Meeker, "Active ear defender systems: Development of a laboratory model," Wright Air Development Division, Air Research and Development Command, U.S. Air Force, Rep., 1959.
- L. J. Fogel, "Apparatus for improving intelligence under high ambient noise levels," U.S. Patent 2 966 549, Ara. 27, 1960. [Çevrimiçi]. Erişim adresi: <https://patents.google.com/patent/US2966549A>.
- H. F. Olson, "Electronic sound absorber," U.S. Patent 2 983 790, May. 9, 1961. [Çevrimiçi]. Erişim adresi: <https://patents.google.com/patent/US2983790A>.
- A. G. Bose ve J. Carter, "Headphoning," U.S. Patent 4 455 675, Haz. 19, 1984. [Çevrimiçi]. Erişim adresi: <https://patents.google.com/patent/US4455675A>.
- "History," Bose Aviation Headsets. [Çevrimiçi]. Erişim adresi: <https://boseaviation.com/about/history/>.
- D. R. Morgan, "An analysis of multiple correlation cancellation loops with a filter in the auxiliary path," IEEE Trans. Acoust., Speech, Signal Process., c. 28, sy 4, ss. 454-467, Ağu. 1980, doi: 10.1109/TASSP.1980.1163430.
- H. Zhang ve D. Wang, "Deep ANC: A deep learning approach to active noise control," Neural Netw., c. 141, ss. 1-10, Eyl. 2021, doi: 10.1016/j.neunet.2021.03.037.
- A. R. Yuliani, M. F. Amri, E. Suryawati, A. Ramdan ve H. F. Pardede, "Speech enhancement using deep learning methods: A review," J. Elektron. dan Telekomun., c. 21, sy 1, ss. 19-26, 2021, doi: 10.14203/jet.v21.19-26.
- G. Wang ve diğerleri, "Low-latency real-time deep learning," IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP) içinde, 2020.
- I. Hubara, Y. Nahshan, Y. Hanani, R. Banner ve D. Soudry, "Accurate post training quantization with small calibration sets," Proc. of the 38th Int. Conf. on Mach. Learn. (ICML) içinde, c. 139, ss. 4466-4475, 2021.
- T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden ve A. Peste, "The state of

sparsity in deep neural networks," arXiv preprint arXiv:1902.09574, 2021.

- A. G. Howard ve diğerleri, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.
- <https://www.kaggle.com/datasets/niranjankn/esc50>
- <https://www.kaggle.com/datasets/chrisfilo/urbansound8k>
- <https://www.kaggle.com/datasets/pypiahmad/librispeech-asr-corpus>